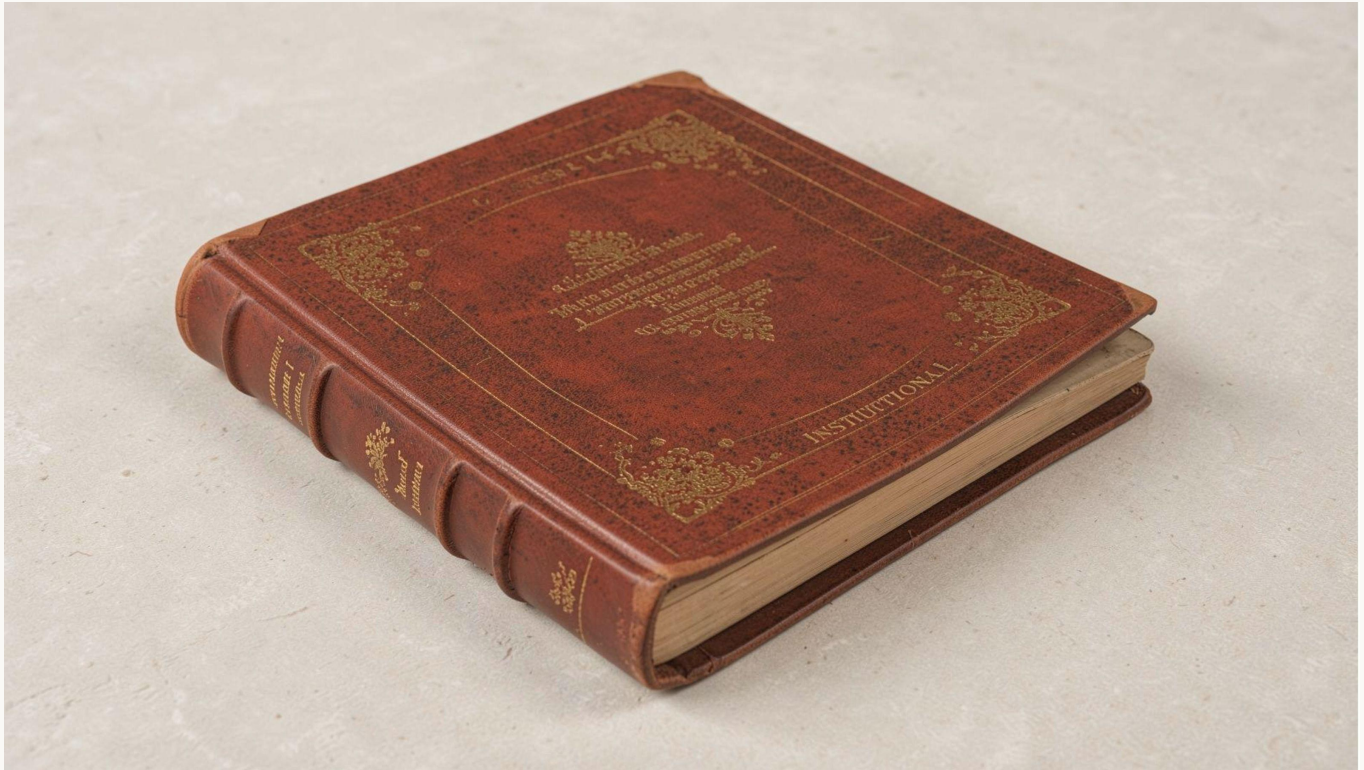




FINAL PROJECT REPORT · APPLIED COMPUTING

Attendance Risk Analyzer

Final Project Report on a web-based system that identifies attendance risk using Python, Flask, SQLite, Greedy methods, Sorting, and Binary Search.

Candidate Details : **L.RAHUL** **MD.IMRAN**
 L.SAITEJA **M.CHARAN TEJ**

Institution Department & College: [AIML , MALLA REDDY TECHNICAL CAMPUS]

Session Academic Year: 2026

Abstract and Introduction

An institutional report on transitioning academic monitoring from manual audit logs to automated risk analytics and deterministic interventions.

Abstract

The Attendance Risk Analyzer is an institutional web platform engineered to capture session attendance, compute accurate running attendance ratios, and classify student cohorts into actionable risk tiers: Safe, Warning, and Critical. Traditional institutions face severe administrative latency caused by paper ledgers and isolated flat spreadsheets. By coupling a lightweight relational datastore with algorithmic ranking and logarithmic lookup pipelines, this solution converts static attendance archives into an active, timely advisory mechanism for faculty, mentors, and departmental authorities.

Proposed solution architecture

The system is built upon a modular Flask micro-framework back end backed by an ACID-compliant SQLite relational persistence layer. The operational core encapsulates strict server-side validation filters, role-based view enforcement, dynamic percentage calculation, and automated classification logic. It delivers instantaneous administrative overviews, actionable risk indicators, and direct student transparency without LMS maintenance overhead.

Introduction & administrative challenge

Institutional attendance compliance across multiple academic departments, elective modules, and batch timetables presents complex coordination challenges. Manual roll-call registers and decentralized spreadsheet entries are inherently error-prone, labor-intensive, and reactive. Administrative staff typically discover attendance shortages at semester-end audits when regulatory thresholds (such as statutory 75% or 80% mandates) have already been violated beyond recovery.

System purpose & operational utility

The central purpose of this utility is to close the diagnostic delay between a student's initial pattern of unexcused absences and institutional intervention. Providing faculty with immediate risk sorting and offering students authenticated personal visibility replaces punitive semester-end disqualifications with proactive mid-semester recovery paths.

Problem Statement, Objectives, and Scope

Evaluating existing operational deficits, establishing targeted system benchmarks, and delineating functional boundaries for the baseline release.

Problem statement

Conventional attendance maintenance in higher education depends on physical sign-in registers or unstructured spreadsheets distributed across individual faculty members. This setup introduces four critical bottlenecks:

- Severe latency between classroom absences and parental or mentor notification.
- Lack of cross-subject aggregation to detect systemic disengagement across simultaneous courses.
- Absence of fast query tools to isolate students falling below mandatory minimum attendance rules.
- Excessive administrative overhead required to manually verify semester examination eligibility.

Target stakeholder profiles

The primary users comprise course instructors logging daily or period records, academic mentors monitoring student cohorts, department heads assessing systemic risk, and students auditing personal records.

Core project objectives

- **Reliable Persistence:** Store and maintain student profiles, subject registries, timetable periods, and session records without duplication.
- **Automated Computation:** Continuously compute aggregated and course-specific percentage scores through deterministic rules.
- **Risk Stratification:** Classify every registered student into defined Safe, Warning, and Critical cohorts against configurable statutory thresholds.
- **Algorithmic Speed:** Provide quick sorting mechanisms and binary search queries for instant record inspection.
- **Access Security:** Enforce role-based access control alongside rigorous input sanitization and verification.

System boundary & scope exclusions

The initial release targets attendance tracking, risk computation, and academic alert preparation. It specifically excludes broad learning management workflows, examination grading frameworks, tuition billing, and hostel management.

Features and Functional Requirements

A concise specification of system capabilities, user responsibilities, security, and operating requirements.

Main features

- Secure login and logout for authorized users.
- Student and subject master-data management.
- Daily or period-wise attendance entry.
- Automatic percentage and risk calculation.
- Dashboard filters by class, subject, semester, and risk level.
- Sorted student rankings and fast student lookup.
- Attendance reports and export-ready summaries.

User functionalities

Administrators manage users and master data; faculty record and review attendance; students view their own attendance and risk status.

CRUD operations

Create, read, update, and delete student, subject, attendance-session, and user records; require confirmation for destructive actions.

Authentication and validation

Require authenticated sessions; validate required fields and numeric ranges; prevent duplicate records; sanitize input; and restrict users to permitted actions.

System requirements

A modern browser, Python 3.x runtime, Flask, SQLite, a local or hosted server, and basic hardware capable of running the application and database.

Algorithms and Methodology

Formulation of core analytical logic: deterministic percentage derivation, greedy intervention allocation, sorting orders, and logarithmic search.

Mathematical formulation

For any student in course i with attended sessions A_j and total conducted sessions T_j , the attendance percentage P_j is calculated as:

$$P_j = \left(\frac{A_j}{T_j}\right) \times 100 \quad \text{where } T_j > 0$$

Given required threshold θ (e.g., 75%), the sessions needed to restore compliance without further absences, denoted R_j , follows:

$$R_j = \max\left(0, \left\lceil \frac{\theta \cdot T_j - 100 \cdot A_j}{100 - \theta} \right\rceil \right)$$

Greedy intervention strategy

Faculty advising capacity is resource-constrained. When allocating mentor hours across n non-compliant students, the system applies a greedy approach based on the chosen institutional policy:

- **Maximum Deficit First:** Prioritize students with lowest P_j to mitigate severe disengagement.
- **Minimal Recovery Effort First:** Prioritize students with lowest positive R_j , maximizing the total number of students brought across the threshold under tight time limits.

Sorting & binary search pipelines

Tabular rosters and prioritized lists require efficient order structuring:

- **Timsort Optimization:** Python's hybrid merge-insertion sort orders cohorts by percentage, required recovery sessions, or student identifier in worst-case time with best-case performance on pre-ordered records.
- **Binary Search Lookup:** Once records are indexed by unique identifier (roll number) or threshold value, exact student search and tier boundary location execute in comparisons, avoiding linear table scans across large student bodies.

Data flow methodology

The project followed an agile prototype lifecycle: database schema definition, Flask routing integration, algorithmic test-case verification, responsive dashboard assembly, and boundary-condition stress testing.

Technologies and Implementation

Layer	Stack and role
Frontend	HTML5, CSS3, Bootstrap or a comparable responsive framework, and JavaScript for forms, tables, filters, and dashboard interactions.
Backend	Python with Flask for routing, sessions, request handling, validation, business logic, and server-side rendering or JSON endpoints.
Database	SQLite relational database with users, students, subjects, attendance_sessions, attendance_records, and risk_settings tables.
Tools	An IDE, Git, a virtual environment, Flask development tools, SQLite utilities, and optional charting or export libraries.

Implementation details

- Calculate attendance percentage as attended classes divided by total classes multiplied by 100.
- Store risk thresholds as configurable values, not hard-coded across multiple routes.
- Separate presentation, route handling, database access, and algorithm modules.
- Use parameterized queries and clear error messages.
- Return responsive tables and summary cards for high-risk, warning, and safe groups.

Results, Testing, and Performance

The prototype can register users, manage attendance data, calculate percentages, identify risk groups, search records, and present actionable summaries. Replace this narrative with measured results after deployment.

Test cases

- Valid and invalid login credentials.
- Student creation with missing, duplicate, or invalid fields.
- Attendance entry with valid and out-of-range values.
- Percentage calculation for zero, partial, and complete attendance.
- Risk classification at and around each threshold.
- Sorting in ascending and descending order.
- Binary search for an existing and a missing student.
- Unauthorized access to restricted routes.
- Update and delete operations with confirmation and persistence checks.

Testing methods

Use unit tests for calculations and algorithms; integration tests for routes and database operations; validation tests for forms; usability testing with representative users; and security checks for sessions and access control.

Algorithmic complexity

Sorting generally costs $O(n \log n)$; binary search costs $O(\log n)$ on sorted data; and a full scan costs $O(n)$.

Performance analysis

Record response time, dataset size, and query behavior using a representative test dataset.

Metric	Value to record
Dataset size	[Enter count]
Average route response time	[Enter ms]
Batch attendance write time	[Enter ms]
Sorting & binary search pass	[Enter ms]
Query behavior	[Describe]

Screenshot and output evidence

Insert screenshots of the login page, dashboard, attendance-entry form, risk summary, sorted report, and student-search result. Include sample output tables or charts where available.

Conclusion, Future Scope, and References

Conclusion, proposed enhancements, and project references.

Conclusion

The Attendance Risk Analyzer converts attendance records into early warnings. Its simple web stack and core algorithms support maintainability, efficient lookup, and informed intervention.

References and documentation

- Python Documentation — <https://docs.python.org/3/>
- Flask Documentation — <https://flask.palletsprojects.com/>
- SQLite Documentation — <https://www.sqlite.org/docs.html>
- Bootstrap Documentation — <https://getbootstrap.com/docs/>
- Standard algorithm and data-structures textbook or course notes used by the project team.

Future enhancements

- Mobile-friendly or progressive web application support.
- Email, SMS, or in-app notifications for risk alerts.
- Predictive models for absence forecasting.
- Role-based analytics for parents, mentors, and department heads.
- Cloud deployment, automated backups, and audit logs.
- Integration with student information or learning-management systems.
- Charts for attendance trends, subject comparisons, and recovery planning.

Report compiled for institutional evaluation · Academic Year [Enter academic year]